



A Pipeline for Visualising MATSim Events in Blender

Daniel Steven Bell

Newcastle University, UK

daniel.bell2@ncl.ac.uk

Motivation

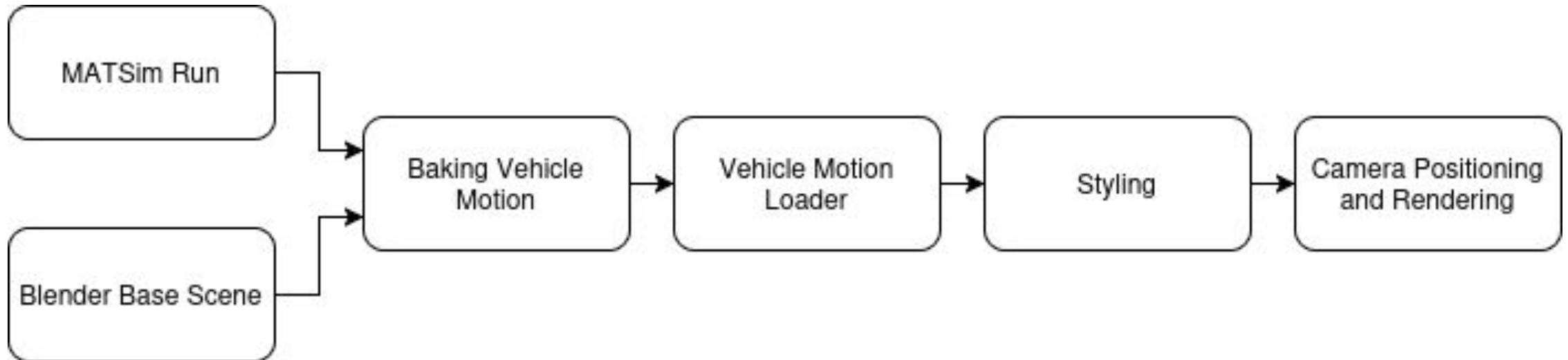
1. Existing tools provide strong interactive and technical analysis (validating a model, interrogating results and debugging), but are not designed for immersive or cinematic 3D presentation.
2. Public and stakeholder-facing communication.
3. General-purpose 3D tools are increasingly used for scientific/simulation visualisation — an opportunity to bring this to transport.

What is Blender?

- 3D Modelling and animation tool
- Open source
- Active community
- Powerful tooling and render engines
- Extensive catalogue of add-ons
- BlenderGIS add-on -
<https://github.com/domlysz/blendergis>
- MATSim compatibility gap:
 - Incompatible with xml data
 - Struggles with a large number of individual objects

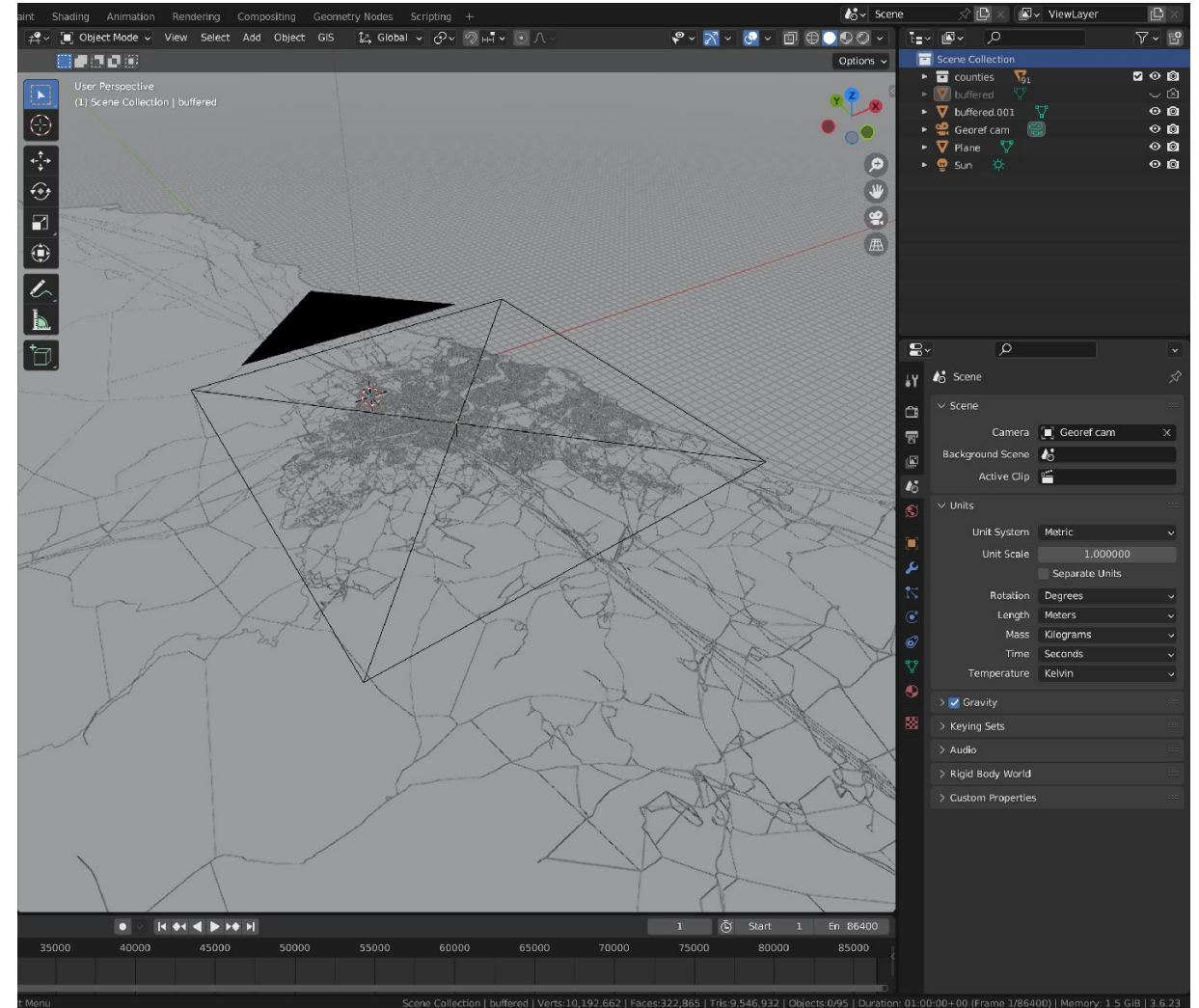


Pipeline



1. Scene Setup and MATSim Outputs

- Create a new scene in Blender:
 - An existing file from which to run Blender in headless mode (no GUI)
 - To embed vehicle motion script
 - Template scene with preferred GIS layers, styling, and render settings
 - Must have BlenderGIS add-on installed
- MATSim Run:
 - 319,841 unique links, 125,706 unique nodes, 142,272 unique vehicles

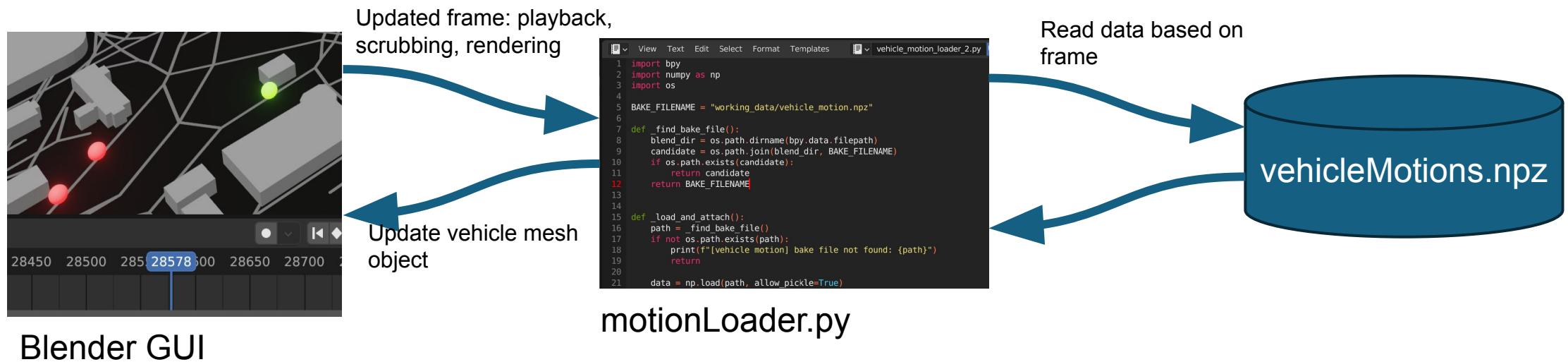


2. Baking Vehicle Motion

- Load Network Nodes (network.xml)
 - Transform node locations to Blender local scene coordinate system (including elevation) in the form of a shapefile, using BlenderGIS
 - Write to disk as a numpy array
 - Slowest step, but only needs to be done once per network
- Parsing MATSim events (events.xml)
 - Vehicle enters traffic, vehicle leaves traffic, entered link, left link
 - Reference event link IDs to new node coordinates, and calculates position for each frame
 - Save vehicle motions to a numpy array (vehicle IDs, frames, positions, visibility, mode)
- Create an empty Blender mesh object in the scene
- Embed a python script within Blender to read vehicle motion array

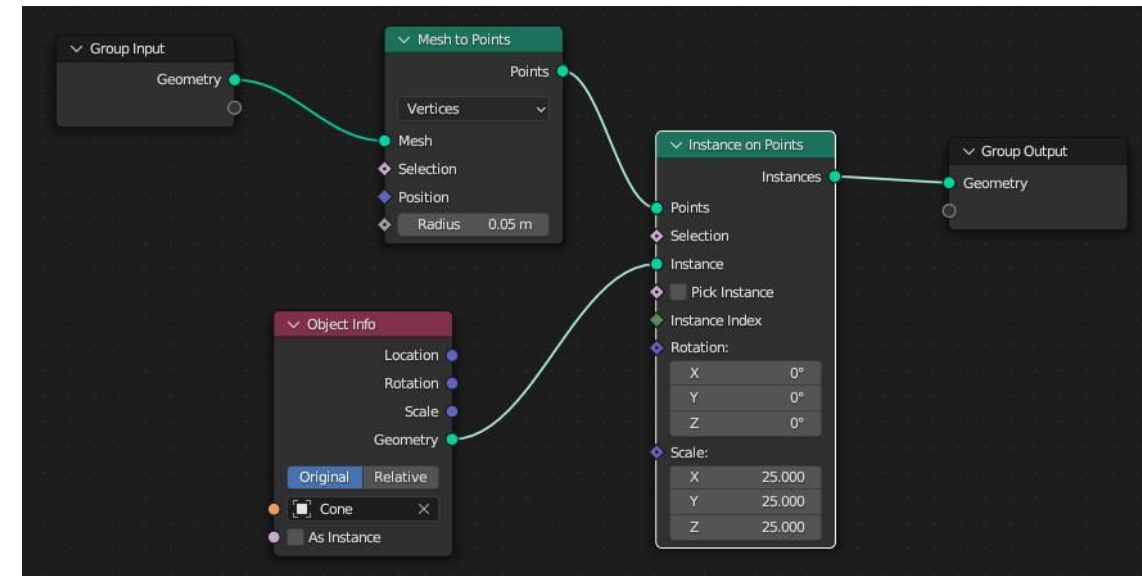
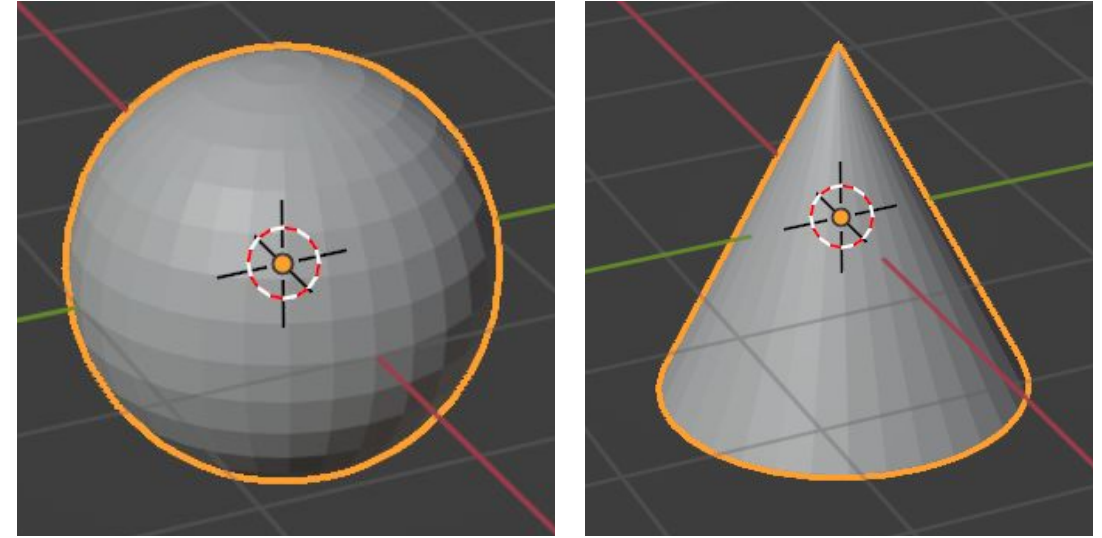
3. Vehicle Motion Loader

- Applies vehicle positions to the empty mesh object created earlier
- Function registered as a frame handler
- Vehicle direction and speed calculated at this stage on the fly



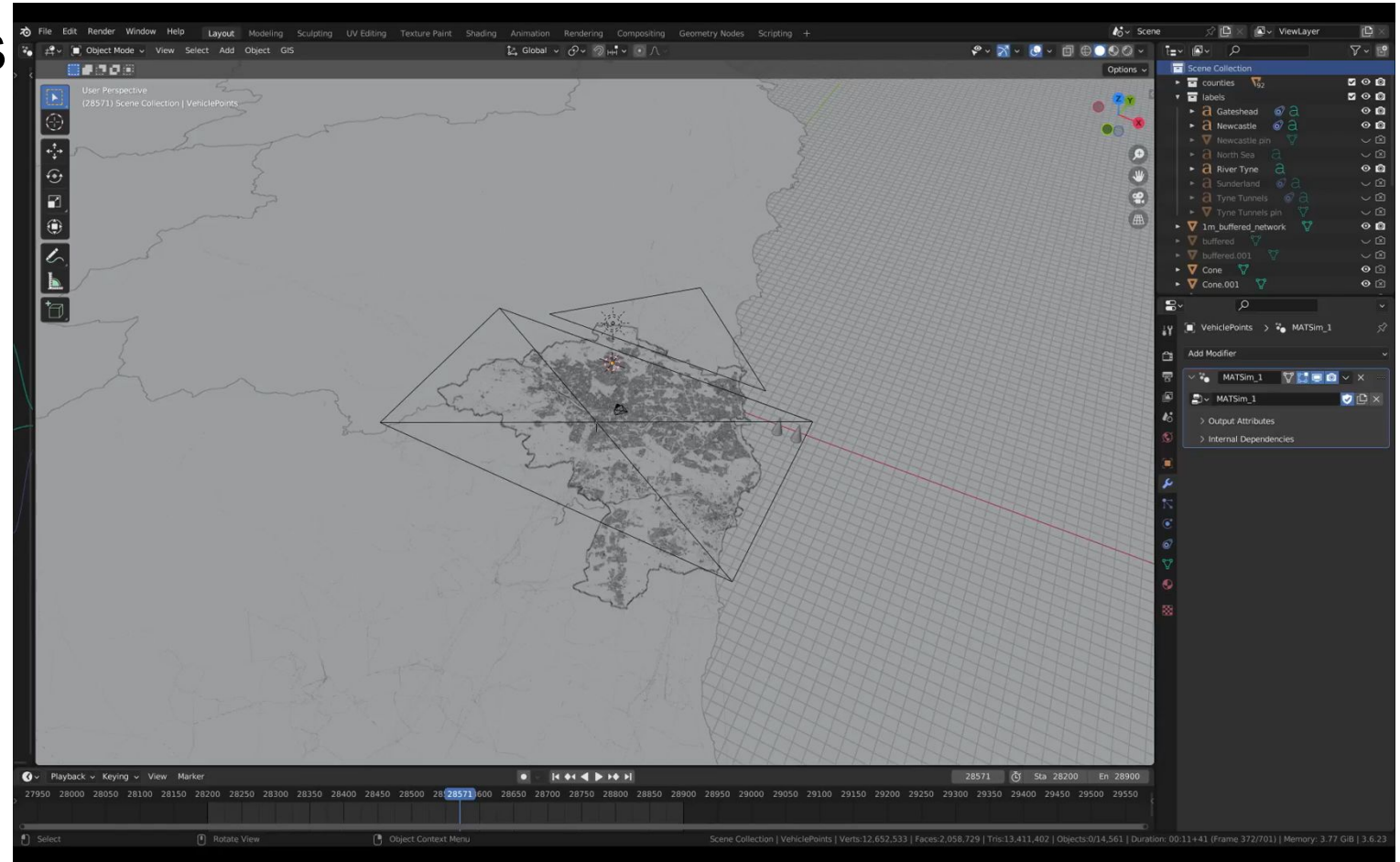
4. Styling

- Apply a geometry to each vehicle point using a template mesh, using Geometry Nodes
 - Simple shapes: cube, sphere, cone
 - Custom and community made models: cars, buses, bicycles
- Rotation based on direction attribute
- Shader editor



5. Rendering

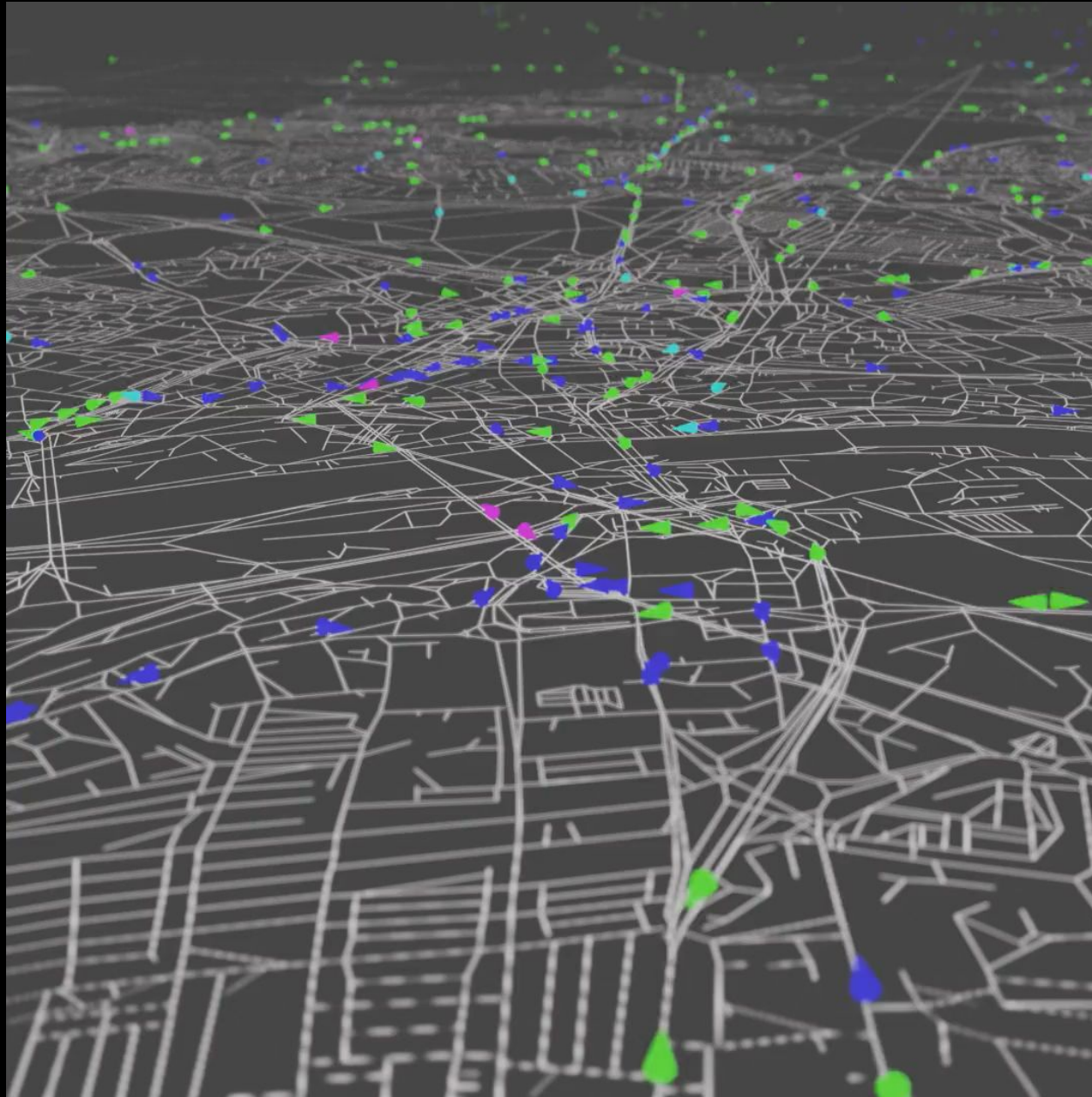
- Edit scene, GIS layers
- Camera positioning
- Lighting effects
- Resolution
- Sampling
- Frame range
- Frame step
- Render time
- Render engine





visuals
DARe

<https://visuals.dare.ac.uk/demos>



<https://visuals.dare.ac.uk/demos>





Tyne and Wear Extreme Flooding Event

08:00



Questions and Feedback

Daniel Steven Bell

Newcastle University, UK

daniel.bell2@ncl.ac.uk

<https://visuals.dare.ac.uk/demos>